



Bahria University, Islamabad Campus

Department of Computer Science & Information Technology

CEN 323 — Computer Organisation & Assembly Language (COAL)

SEMESTER PROJECT

FINAL IMPLEMENTATION & DOCUMENTATION

MAZE RACER

A First-Person 3D Maze Game Written Entirely in x86 Assembly

Course Instructor: Adnan Jelani

Group Number: 5

Program: BS Information Technology — 6th Semester

Member Name	Registration No.	Role
Muhammad Tayyaab Zahoor	01-135232-070	Game Logic & Systems
Anood Tayyeba Imtiaz	01-135232-010	Rendering & World

Submission Date: May 17, 2026

Bahria University, E-8, Islamabad, Pakistan

Contents

Abstract	3
1 Introduction	3
1.1 Problem Statement	3
1.2 Motivation	3
1.3 Scope	3
1.4 Objectives	4
2 System Design	4
2.1 Module Architecture	4
2.2 Memory Layout	5
2.3 Game State Machine	5
2.4 Design Decisions	6
3 Implementation Details	6
3.1 Assembly Concept Coverage	6
3.2 Startup and Interrupt Hooking	7
3.3 Main Game Loop	7
3.4 3D Raycasting Engine — <code>render_frame</code>	8
3.4.1 Double-Buffering and VSYNC Blit	8
3.5 Procedural Maze Generation — <code>generate_maze</code>	9
3.5.1 Recursive Backtracker	9
3.5.2 Braid Post-Processing	9
3.6 Linear Congruential Generator — <code>random_step</code>	10
3.7 Dual-Agent Enemy AI — <code>move_enemy</code>	10
3.7.1 Smart Wanderer (type 0)	10
3.7.2 Wall Follower / Hunter (type 1)	10
3.8 Timer Interrupt ISR — <code>timer_isr</code>	11
3.9 PC Speaker Proximity Audio	12
3.10 Hard Mode Fog of War	12
3.11 Scalable Pixel Font — <code>draw_char</code> / <code>draw_string_centered</code>	13
4 CCA Mapping	13
4.1 CCA1 — Complex Problem-Solving	13
4.2 CCA4 — Design Constraints	14
4.3 CCA8 — Team Collaboration	14
5 Testing and Results	14
5.1 Functional Test Cases	14
5.2 Screenshots	16
6 Team Contributions	17
7 Challenges and Lessons Learned	17
7.1 Fixed-Point Arithmetic Without an FPU	17
7.2 INT 1Ch Interrupt Chaining	17
7.3 Segment Register Management	18

7.4	Enemy AI Edge Cases	18
8	Conclusion	18
8.1	Summary of Achievements	18
8.2	Limitations	18
8.3	Future Work	18
	References	19

Abstract

Maze Racer is a fully playable first-person 3D maze game implemented entirely in x86 Assembly Language, assembled with NASM and executed in DOSBox. The game renders an interactive 3D perspective of a procedurally generated maze using a Digital Differential Analyzer (DDA) raycasting engine operating in VGA Mode 13h (320×200, 256-colour). A custom double-buffering scheme writes every frame to an off-screen memory segment before blitting it to VGA memory during the vertical blanking interval, eliminating screen tearing. Maze generation relies on a recursive backtracker algorithm driven by a hardware-seeded linear congruential generator; a braid post-processing pass removes selected walls to prevent dead-end loops in enemy behaviour. Two independent AI agents patrol the maze simultaneously using a wall-following and a smart-wander strategy. Three difficulty modes govern map size, timer budget, enemy visibility, and a fog-of-war minimap. Hardware integration is achieved through INT 1Ch timer interrupt hooking, non-blocking INT 16h keyboard polling, and direct port access to the Programmable Interval Timer for dynamic proximity audio. The result is a technically substantive and playable game that concretely covers the breadth of the CEN 323 curriculum.

1 Introduction

1.1 Problem Statement

Most x86 assembly exercises demonstrate a single concept in isolation: a sorting routine, an interrupt handler, or a string procedure. While useful, these exercises do not require the subsystems to interact, which is where the real complexity of low-level programming lies. Maze Racer addresses this gap by integrating memory-mapped graphics, interrupt-driven timing, hardware port I/O, procedural content generation, and concurrent AI into one unified program that can be played and demonstrated live.

1.2 Motivation

A game provides a natural and immediate feedback mechanism for every assembly concept it uses. When a wall darkens with distance, a bitwise shift-right is rendered visibly. When the PC speaker accelerates its beeping, a port write becomes audible. When the enemy moves at a fixed speed regardless of how long the renderer takes, interrupt decoupling becomes tangible. Every feature in this project is a COAL concept made perceptible in real time.

1.3 Scope

The project implements the following subsystems:

- First-person 3D raycasting renderer in VGA Mode 13h with distance-based shading and double buffering.
- Procedural maze generation via recursive backtracking with braid post-processing.
- Two distinct AI agents (smart wanderer and right-hand-rule wall follower) operating simultaneously.
- Hardware timer interrupt (INT 1Ch) for game clock and enemy movement scheduling.
- Non-blocking keyboard input via INT 16h.

- PC speaker proximity audio via direct port I/O (ports 42h, 43h, 61h).
- Three difficulty tiers: Easy, Normal, and Hard (including fog-of-war and aesthetic changes).
- A scalable custom pixel-font system for all on-screen text rendering.

Leaderboard persistence and an animated pixel-art title screen were identified as out-of-scope after the proposal stage and are noted as future work.

1.4 Objectives

1. Build a fully playable game in 8086 Assembly targeting the real DOS hardware model.
2. Implement a raycasting engine using only integer arithmetic (no FPU instructions).
3. Use INT 1Ch to decouple game-logic speed from rendering speed.
4. Implement procedural maze generation with a hardware-seeded RNG.
5. Provide two independent enemy AI behaviours operating concurrently in the same maze.
6. Deliver three difficulty tiers with measurably distinct gameplay experiences.

2 System Design

2.1 Module Architecture

Figure 1 illustrates the top-level module architecture. All code is contained in a single flat binary (`maze.asm`) with the screen-management routines separated into `screens.asm` and included via NASM's `%include` directive. The design follows a synchronous main-loop model with one asynchronous entry point (the timer ISR).

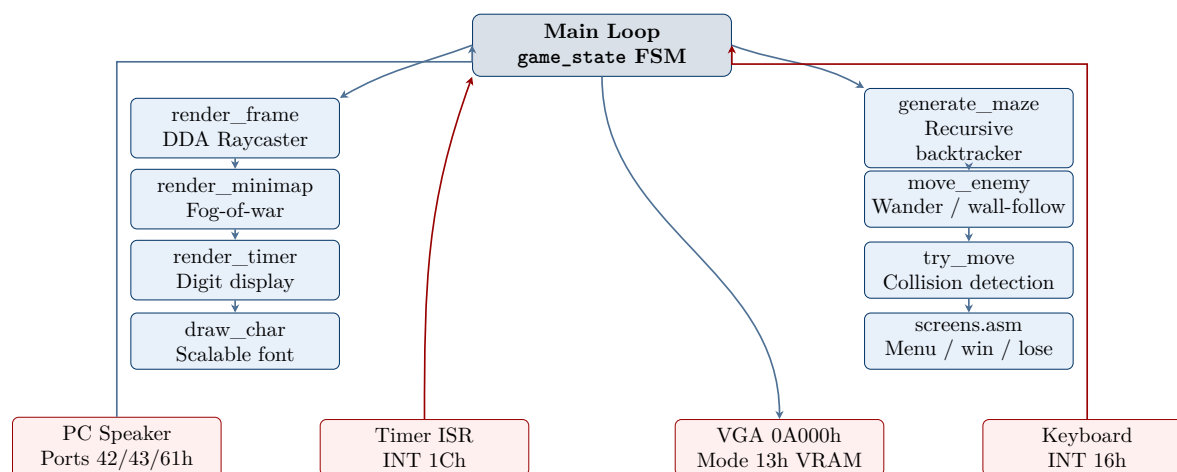


Figure 1: Module architecture of Maze Racer. Blue nodes are software modules; red nodes are hardware interfaces.

2.2 Memory Layout

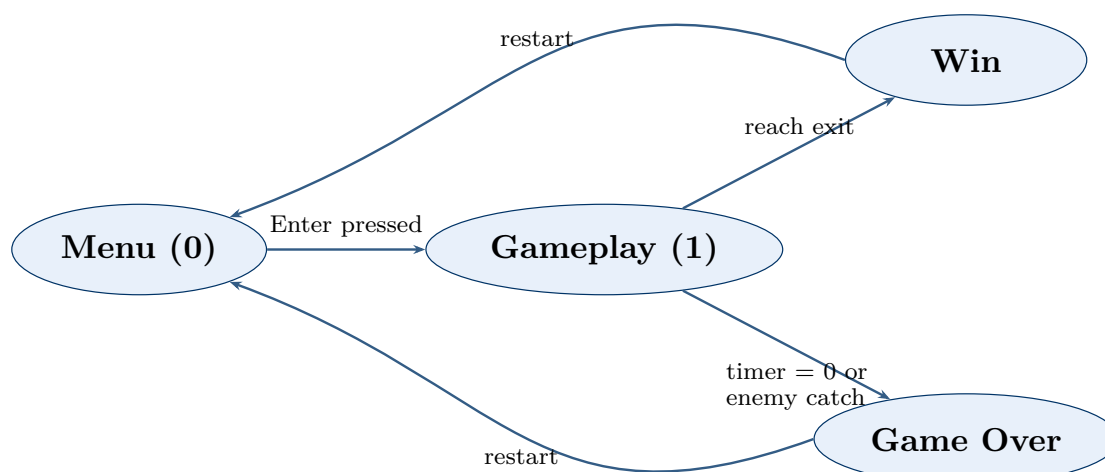
Table 1: Memory layout of Maze Racer

Region	Address / Segment	Contents
<code>.text</code>	CS:0100h	All executable procedures; <code>screens.asm</code> included inline
Double Buffer	CS + 0x1000	64,000-byte off-screen framebuffer (320×200×1)
VGA VRAM	0A000h:0000h	Live display memory; written only during VSYNC blit
<code>.data</code>	CS-relative	Font bitmaps (5 bytes/char), string literals, <code>dir_masks</code> , <code>dir_dx/dy</code>
<code>.bss</code>	CS-relative	<code>maze[225]</code> , <code>visited[225]</code> , <code>explored[225]</code> , <code>m_stack[450]</code> , enemy arrays, player state, render temporaries, seed (4 bytes), old ISR vector

Each of the 225 maze cells (15×15 or 9×9 for Easy mode) is stored as a single byte. Bits 0–3 encode the four walls: bit 0 = North, bit 1 = East, bit 2 = South, bit 3 = West. A set bit means the wall is present; clearing it carves an opening.

2.3 Game State Machine

Program flow is governed by a single byte, `game_state`:

Figure 2: Game state machine for `game_state`.

2.4 Design Decisions

Fixed-point arithmetic. The 8086 has no hardware FPU. All distances, ray vectors, and wall heights use 16-bit integers scaled by 256. Division is performed with `IDIV`; efficient halving uses `SHR 1`. This choice shapes the entire raycaster design.

Tile-based movement. Player and enemy positions are integer cell coordinates, not sub-tile offsets. This reduces collision detection to a single bitmask test on the cell byte.

Interrupt-driven enemy scheduling. Enemies are moved inside the `INT 1Ch` ISR, every 18 ticks (≈ 1 second), keeping their speed constant regardless of renderer load.

Double buffering. Writing to a segment distinct from VRAM and blitting only during `VSYNC` eliminates tearing without requiring a dedicated flip interrupt.

3 Implementation Details

3.1 Assembly Concept Coverage

Table 2 maps the COAL lab topics to specific sections of the implementation.

Table 2: Assembly concept coverage across lab topics

#	Lab Topic	Where Applied in Maze Racer	Key Instructions
1	Data Move-ment	Player/enemy coordinate reads and writes; <code>maze[]</code> array access via <code>DS:SI</code>	<code>MOV</code> , <code>LEA</code> , <code>LDS</code>
2	Arithmetic	Fixed-point ray distance: <code>800 / t_hit</code> ; timer ticks to seconds conversion	<code>IMUL</code> , <code>IDIV</code> , <code>DIV</code> , <code>ADD</code> , <code>SUB</code>
3	Bitwise Logic	Wall bitmask tests; font pixel extraction; LCG seed update; shading select	<code>AND</code> , <code>OR</code> , <code>XOR</code> , <code>SHL</code> , <code>SHR</code> , <code>TEST</code>
4	Branching	Collision check, DDA step selection, AI direction priority, difficulty branches	<code>CMP</code> , <code>JE</code> , <code>JNE</code> , <code>JG</code> , <code>JL</code> , <code>JNS</code>
5	Loops	Column render loop (320 iterations); minimap cell scan; font row/column loops	<code>LOOP</code> , <code>DEC+JNZ</code> , <code>REP STOSB/MOVSW</code>
6	Procedures	Every subsystem is a named procedure with full push/pop register discipline	<code>CALL</code> , <code>RET</code> , <code>PUSH</code> , <code>POP</code>
7	Stack Operations	Software-stack maze generator; ISR register save/restore; nested procedure calls	<code>PUSH</code> , <code>POP</code> , <code>PUSHF</code> , <code>POPF</code>
8	Arrays & Memory	<code>maze[225]</code> , <code>explored[225]</code> , <code>m_stack[]</code> , direction lookup tables; base+offset addressing	<code>MOV [base+SI], MOV BX, [array+BX]</code>
9	Interrupts	<code>INT 1Ch</code> hooked for timer/enemy; <code>INT 16h</code> for input; <code>INT 10h</code> for mode switch; <code>INT 1Ah</code> for RTC seed	<code>INT</code> , <code>IRET</code> , <code>DOS 21h</code> (25h/35h)
10	Segmented Memory	<code>ES</code> →double buffer during draw; <code>DS</code> ↔ <code>CS</code> in ISR; <code>ES</code> →0A000h for blit; VRAM via segment override	<code>MOV ES/DS, PUSH CS / POP DS</code>

Continued on next page

#	Lab Topic	Where Applied in Maze Racer	Key Instructions
11	String Operations	64,000-byte VSYNC blit; buffer clear; fog-of-war minimap wipe	REP MOVSW, REP STOSB, REP STOSW
12	Hardware I/O	PC speaker frequency and gate; VSYNC poll; VGA VRAM direct write	IN, OUT (ports 03DAh, 42h, 43h, 61h)

3.2 Startup and Interrupt Hooking

On entry the program seeds the LCG random number generator from the real-time clock via INT 1Ah, which reads the hardware timer ticks-since-midnight into CX:DX. It then saves the original INT 1Ch vector using DOS function 35h, installs `timer_isr` using function 25h, and switches to VGA Mode 13h.

```

1      ; Seed LCG from hardware RTC
2      mov ah, 00h
3      int 1Ah                      ; CX:DX = timer ticks since midnight
4      mov word [seed], dx
5      mov word [seed+2], cx
6
7      ; Save old INT 1Ch vector
8      mov ax, 351Ch
9      int 21h                      ; ES:BX = current vector
10     mov [old_int1c_off], bx
11     mov [old_int1c_seg], es
12
13     ; Install our ISR
14     mov ax, 251Ch
15     mov dx, timer_isr
16     int 21h                      ; DS:DX = new handler address
17
18     ; Switch to Mode 13h (320x200, 256 colours)
19     mov ax, 0013h
20     int 10h

```

Listing 1: RNG seeding from hardware clock and ISR installation

3.3 Main Game Loop

Each frame the main loop executes in strict order:

1. Clear the double buffer with sky colour (top 16,000 words) and floor colour (bottom 16,000 words) using `rep stosw`.
2. Check `timer_ticks` against zero; branch to `game_over_time` if expired.
3. Compare player coordinates against each enemy pair; branch to `game_over_enemy` on match.
4. Compare player coordinates against (`map_bound`, `map_bound`); branch to `game_win` on match.
5. Call `render_frame`, `render_minimap`, threat indicator, and `render_timer`.
6. Spin-wait on VGA port 03DAh for VSYNC start; blit 64,000 bytes with `rep movsw`.
7. Poll INT 16h (function 01h); if a key is pending, consume it (function 00h) and dispatch WASD.

3.4 3D Raycasting Engine — `render_frame`

For each of the 320 screen columns, a ray direction vector is derived from the player's facing and the column's offset from the screen centre. Both ray components are fixed-point integers scaled by 256.

```

1      mov ax, [col]
2      sub ax, 160          ; offset from screen centre (-160 to +159)
3      imul ax, 8           ; scale up
4      mov cx, 5
5      cwd
6      idiv cx              ; rx_base = (col - 160) * 8 / 5

```

Listing 2: Per-column ray direction calculation

The DDA then marches the ray through the maze grid, advancing along the axis whose next crossing is nearer. At each grid boundary the cell's bitmask byte is read and tested against the wall bit for that crossing direction. On a wall hit, wall height is:

$$wall_height = \min\left(\frac{800}{t_hit}, 200\right)$$

Distance-based shading selects between two VGA colour indices using a simple threshold on the normalised distance:

```

1      mov ax, [t_hit]
2      shr ax, 4            ; normalise: divide by 16
3      mov bx, ax
4      mov al, 7            ; close wall: light grey
5      cmp bx, 3
6      jl .color_done
7      mov al, 8            ; far wall: dark grey
8  .color_done:
9      ; Hard mode overrides with orange palette entries 42/43
10     cmp byte [difficulty], 2
11     jne .save_color
12     mov al, 42
13     cmp bx, 3
14     jl .save_color
15     mov al, 43
16  .save_color:
17     mov [wall_color], al

```

Listing 3: Distance-based wall shading via SHR

3.4.1 Double-Buffering and VSYNC Blit

All drawing targets a 64 KB segment at `CS + 0x1000`. After each frame the code waits for the vertical blanking interval on VGA port 03DAh then copies the full 64,000-byte buffer to VRAM atomically:

```

1      mov dx, 03DAh
2  .vsync_wait1:
3      in al, dx
4      test al, 8
5      jnz .vsync_wait1    ; spin while in retrace

```

```

6 .vsync_wait2:
7   in  al, dx
8   test al, 8
9   jz  .vsync_wait2      ; spin until retrace begins
10  ; --- blit 64000 bytes ---
11  push ds
12  mov ax, cs
13  add ax, 0x1000
14  mov ds, ax            ; DS = off-screen buffer
15  mov ax, 0A000h
16  mov es, ax            ; ES = VGA VRAM
17  xor si, si
18  xor di, di
19  cld
20  mov cx, 32000
21  rep movsw             ; 32000 word transfers = 64000 bytes
22  pop ds

```

Listing 4: VSYNC-synchronised double-buffer blit

3.5 Procedural Maze Generation — generate_maze

3.5.1 Recursive Backtracker

All 225 cells are initialised to 0x0F (all four walls present). A software stack (`m_stack`, a word array in BSS) replaces hardware recursion:

```

1 .push_neighbor:
2   mov byte [visited + bx], 1      ; mark neighbour visited
3   mov di, [m_sp]
4   mov [m_stack + di], bx         ; push cell index
5   add word [m_sp], 2
6   jmp .gen_loop
7
8 .no_unvisited:                    ; dead end -- backtrack
9   sub word [m_sp], 2
10  jmp .gen_loop

```

Listing 5: Software-stack push/backtrack in the recursive backtracker

When a passage is carved, the corresponding wall bits on *both* sides of the boundary are cleared with AND:

```

1 .rem_n:
2   and byte [maze + bx], 0FEh      ; clear North wall of current cell
3   sub bx, 15
4   and byte [maze + bx], 0FBh      ; clear South wall of neighbour

```

Listing 6: Bilateral wall removal (carving North example)

3.5.2 Braid Post-Processing

After generation the perfect maze is a tree (exactly one path between any two cells). An additional loop randomly removes internal walls to create loops and alternative routes, preventing the wall-following AI from cycling:

```

1      mov cx, 20                ; knock down 20 additional internal walls
2  .braid_loop:
3      push cx
4      call random_step
5      xor dx, dx
6      mov cx, 225
7      div cx
8      mov bx, dx                ; pick random cell 0..224
9      call random_step
10     xor dx, dx
11     mov cx, 4
12     div cx                    ; pick random direction 0..3
13     ; apply AND mask to remove that wall on both sides
14     pop cx
15     loop .braid_loop

```

Listing 7: Braid post-processing: random wall removal

3.6 Linear Congruential Generator — random_step

The hardware-seeded LCG produces a new 32-bit pseudo-random value each call, returning the high word in AX. The multiplier and addend are standard Numerical Recipes constants:

```

1  random_step:
2      mov ax, [seed]
3      mov cx, 4E6Dh
4      mul cx                    ; low word of (seed_low * mult)
5      push ax
6      push dx
7      ; ... combine with upper seed word, add 0x3039
8      mov [seed], ax
9      mov [seed+2], dx
10     mov ax, dx                ; return high word
11     ret

```

Listing 8: 32-bit LCG RNG

3.7 Dual-Agent Enemy AI — move_enemy

Two enemies share the same update procedure, dispatched by `enemy_type`. Both run once per second via the timer ISR.

3.7.1 Smart Wanderer (type 0)

Collects all open directions at the current cell, removes the 180° reverse direction unless it is the only option, then selects randomly among the remainder. This prevents the back-and-forth oscillation of a purely random mover while still being non-deterministic.

3.7.2 Wall Follower / Hunter (type 1)

Implements the right-hand rule. At each step it attempts directions in the following priority order using modulo-3 arithmetic on the current heading:

```

1  .wall_follower:
2      mov al, [enemy_dir + si]
3      inc al
4      and al, 3                      ; 1. try right turn (heading + 1) mod 4
5      call check_dir_open
6      cmp ah, 1
7      je  .wf_open
8
9      mov al, [enemy_dir + si] ; 2. try straight ahead
10     call check_dir_open
11     cmp ah, 1
12     je  .wf_open
13
14     mov al, [enemy_dir + si]
15     add al, 3
16     and al, 3                      ; 3. try left turn (heading + 3) mod 4
17     call check_dir_open
18     cmp ah, 1
19     je  .wf_open
20
21     mov al, [enemy_dir + si]
22     add al, 2
23     and al, 3                      ; 4. reverse (only if completely
                                   enclosed)
24 .wf_open:

```

Listing 9: Right-hand wall-follower direction priority

3.8 Timer Interrupt ISR — `timer_isr`

The ISR fires ≈ 18.2 times per second. It saves all registers (including flags via `PUSHF`), switches DS to CS for variable access, decrements `timer_ticks`, calls `move_enemy` every 18 ticks, then chains to the original BIOS ISR via a far jump:

```

1  timer_isr:
2      pushf
3      push ax
4      push bx
5      push cx
6      push dx
7      push si
8      push di
9      push ds
10     push es
11     push cs
12     pop  ds                      ; DS = CS (access our .bss/.data)
13
14     dec word [timer_ticks]
15
16     mov ax, [timer_ticks]
17     mov cx, 18
18     xor dx, dx
19     div cx
20     cmp dx, 0
21     jne .skip_enemy
22     call move_enemy              ; move enemies once per second

```

```

23 .skip_enemy:
24     pop es
25     pop ds
26     pop di
27     pop si
28     pop dx
29     pop cx
30     pop bx
31     pop ax
32     popf
33     jmp far [cs:old_int1c_off] ; chain to original BIOS handler

```

Listing 10: Timer ISR with register discipline and ISR chaining

3.9 PC Speaker Proximity Audio

In Hard mode, when the Manhattan distance between the player and the Hunter enemy falls below five tiles, the Programmable Interval Timer is programmed directly through port writes and the speaker gate is enabled. Pitch is inversely proportional to distance:

```

1 ; cl = Manhattan distance to Hunter (0-4)
2 mov al, 0B6h
3 out 43h, al ; PIT command byte: ch2, LSB+MSB, square
  wave
4 mov ax, 2000 ; base frequency divisor
5 mov bl, cl
6 mov bh, 0
7 imul bx, 500
8 sub ax, bx ; divisor decreases (pitch rises) as cl
  falls
9 out 42h, al ; write low byte of divisor
10 mov al, ah
11 out 42h, al ; write high byte of divisor
12 in al, 61h
13 or al, 3
14 out 61h, al ; set bits 0-1: connect PIT ch2 to
  speaker gate

```

Listing 11: Dynamic PIT-driven proximity beep

A concurrent red strobe border (4-pixel band on all edges) is drawn to the framebuffer when the threat is active, providing a visual warning alongside the audio.

3.10 Hard Mode Fog of War

Each cell the player steps on sets a byte in the `explored[225]` array. The minimap renderer tests this flag before drawing any cell and skips cells not yet explored:

```

1 cmp byte [difficulty], 2 ; Hard mode?
2 jne .not_fow
3 mov ax, [mmap_y]
4 imul ax, 15
5 add ax, [mmap_x]
6 mov si, ax
7 cmp byte [explored + si], 0 ; cell unexplored?
8 je .skip_cell ; do not render it

```

```
9 .not_fow:
```

Listing 12: Fog-of-war visibility check in the minimap renderer

In Hard mode enemies are never drawn on the minimap; the PC speaker is the only warning.

3.11 Scalable Pixel Font — `draw_char` / `draw_string_centered`

Each character is five bytes (five 3-bit-wide rows, packed into the MSBs). The renderer extracts each pixel with `SHL`, then scales the pixel block by `BH` (scale factor passed by caller) by writing a `BH×BH` filled square:

```
1 .col_loop:
2     shl byte [dc_bits], 1      ; shift MSB into Carry flag
3     jnc .skip_pix             ; Carry = 0 => no pixel at this position
4     ; compute top-left (dc_px, dc_py) for this block
5     ; write BH x BH coloured pixels to the double buffer
6 .skip_pix:
7     inc word [dc_x]
8     cmp word [dc_x], 3
9     jl .col_loop
10    inc word [dc_y]
11    cmp word [dc_y], 5
12    jl .row_loop
```

Listing 13: Font pixel extraction and scaled rendering

`draw_string_centered` computes total pixel width as `strlen × scale × 4`, subtracts from 320, and halves the result to find the starting X coordinate before iterating over characters.

4 CCA Mapping

4.1 CCA1 — Complex Problem-Solving

Maze Racer satisfies the Complex Problem-Solving attribute through the non-trivial, tightly coupled interaction of five independent subsystems. The raycaster (`render_frame`) reads the maze bitmask produced by `generate_maze` to determine wall hits; the enemy scheduler (`move_enemy`) reads the same bitmask through `check_dir_open` to evaluate valid moves; the minimap (`render_minimap`) reads the `explored[225]` byte array updated by the main loop after every player step; all rendering targets a shared double buffer that the main loop blits atomically after `VSYNC`; and the timer ISR (`timer_isr`) modifies `timer_ticks` and calls `move_enemy` asynchronously while the main loop executes. The conditional state management spans two execution contexts (synchronous main loop and asynchronous ISR), requiring careful register discipline (`PUSHF/POPF`, nine `PUSH/POP` pairs) and explicit segment switching (`PUSH CS; POP DS`) on every ISR entry. The DDA algorithm requires fixed-point arithmetic across five accumulator variables (`tMaxX/Y`, `tDeltaX/Y`, `t_hit`); maze generation simulates recursion with a software stack; and the AI dispatches two different pathfinding strategies from a single shared loop body. No procedure can be trivially removed without breaking program correctness.

4.2 CCA4 — Design Constraints

The 8086 ISA imposes hard constraints that shaped every design decision. The absence of an FPU required all geometric computation in `render_frame` to use 16-bit fixed-point integers scaled by 256, with wall height computed via `DIV (800 / t_hit)` and shading selected via `SHR 4`. The four general-purpose registers are routinely exhausted within a single procedure: for example, the DDA loop uses `AX` for arithmetic, `BX` for wall masks, `CX` as the bound constant, `DX` as the `IDIV` remainder, `SI` as the maze array index, and `DI` as the VRAM write offset simultaneously. The 16-bit segmented memory model required explicit management throughout: `ES` points to the double buffer during all draw calls and is switched to `0A000h` for the blit; the ISR repoints `DS` to `CS` on entry via `PUSH CS / POP DS`; and the original ISR vector is restored at exit via `LDS DX, [old_int1c_off]` before calling `INT 21h` function `25h`. BIOS and DOS interrupt conventions are observed in full: `INT 21h` functions `25h/35h` for vector management, `INT 10h` for mode switching, `INT 16h` with `AH=01h/00h` for non-blocking key detection, `INT 1Ah` for RTC seeding, and `INT 15h` function `86h` for hardware delays. Direct hardware I/O through `IN/OUT` instructions accesses ports `03DAh` (`VSYNC`), `42h/43h` (`PIT` frequency), and `61h` (speaker gate), with no OS abstraction layer involved.

4.3 CCA8 — Team Collaboration

The project was partitioned along a natural boundary between rendering/world and game-logic/systems.

- **Anood Tayyeba Imtiaz (01-135232-010)** owned: `render_frame` (DDA raycaster, distance shading, buffer draw loop), `render_minimap` (fog-of-war, `explored` array, border drawing), `generate_maze` (recursive backtracker, braid post-processing, LCG integration), the pixel-font subsystem (`draw_char`, `draw_string_centered`, `draw_menu_item`), and VGA Mode `13h` setup/segment management.
- **Muhammad Tayyaab Zahoor (01-135232-070)** owned: `timer_isr` (`INT 1Ch` hook, register discipline, ISR chaining), `move_enemy` (smart wanderer and wall-follower), `try_move` (bitmask collision detection), the PC speaker driver (`play_sound`, `stop_sound`, `beep_jingle`, `beep_descend`), the threat indicator (red strobe border and dynamic `PIT` pitch), `render_timer` (digit extraction/drawing), and `screens.asm` (menu loop, game-over/win screens, difficulty selection, restart logic).

Both members collaborated on integration debugging and the game state machine. Commit history on the GitHub repository reflects independent, distributed contributions from both members throughout the development period; specific commit IDs are listed in Section 6.

5 Testing and Results

5.1 Functional Test Cases

Table 3: Functional test cases for Maze Racer

Test Scenario	Expected Outcome	Actual Outcome	Result
W pressed, facing North, open cell	Player moves North; <code>player_y</code> decrements	Position updated correctly	PASS
W pressed into a wall	Player stays; position unchanged	No movement occurs	PASS
A / D pressed	Facing cycles left / right by 90°	All four headings correct	PASS
S pressed (move backward)	Moves in opposite facing direction	Correct reverse movement	PASS
Player reaches exit tile (<code>map_bound</code> , <code>map_bound</code>)	Win screen; gold flash; jingle	Win triggered correctly	PASS
Timer reaches 0	Red column fill then Game Over screen	Correct animation and screen	PASS
Enemy occupies same cell as player	Immediate Game Over	Triggered on next frame	PASS
Easy mode selected	9×9 map; no enemies; 200 s timer	Correct map size; enemies absent	PASS
Normal mode selected	15×15; 2 enemies visible on minimap; 120 s	Enemies shown on map correctly	PASS
Hard mode selected	15×15; orange walls; fog-of-war; 60 s	All Hard mode features active	PASS
Hard mode – Hunter ≤ 4 tiles	Red strobe border + rising-pitch beep	Strobe and pitch increase confirmed	PASS
Hard mode – Hunter > 4 tiles	Speaker silent; no strobe	Speaker off; no border	PASS
Fog-of-war minimap	Only visited cells render	Unexplored cells correctly hidden	PASS
New game on retry	Fresh unique maze each run	10 runs produced distinct mazes	PASS
Wall-follower AI	Systematic maze traversal (right-hand rule)	AI traces wall correctly over multiple tests	PASS
Smart-wander AI	No immediate 180° reversal at open intersections	Purposeful forward bias confirmed	PASS
ESC during gameplay	Restore text mode; clean DOS exit	INT 10h + INT 20h executes cleanly	PASS
“Try Easier” after loss on Easy	Option not displayed (already at lowest)	Option correctly hidden	PASS

Table 3 — continued

Test Scenario	Expected Outcome	Actual Outcome	Result
“Try Harder” after win on Hard	Option not displayed (already at highest)	Option correctly hidden	PASS

5.2 Screenshots

Screens from the game UI and gameplay.



Figure 3: Main menu with difficulty selection



Figure 4: Easy mode gameplay (3D view + minimap)



Figure 5: Hard mode with explored area only map and threat indicator active



Figure 6: Result screen after game end

6 Team Contributions

Table 4: Team contribution summary

Member Name	Reg. No.	Module(s) Owned
Anood Tayyeba Imtiaz	01-135232-010	<code>screens.asm</code> : menu, game-over, win, difficulty, restart <code>render_minimap</code> (2D grid, fog-of-war, explored array) UI: <code>draw_char</code> , <code>draw_string_centered</code> , <code>draw_menu_item</code> PC speaker driver, <code>render_timer</code>
Muhammad Tayyaab Zahoor	01-135232-070	<code>render_frame</code> (DDA raycaster, shading, double-buffer draw) <code>generate_maze</code> (recursive backtracker, braid, LCG) <code>move_enemy</code> (wall-follower and smart-wander AI) <code>timer_isr</code> (ISR hook), VGA Mode 13h setup, segment management <code>try_move</code> (bitmask collision), threat indicator

GitHub Repository: <https://github.com/TayyaabZ/Maze-Racer>

7 Challenges and Lessons Learned

7.1 Fixed-Point Arithmetic Without an FPU

The DDA algorithm is inherently real-valued. Representing ray vectors, step deltas, and accumulated distances as 16-bit signed integers scaled by 256 required re-deriving every formula to avoid intermediate overflow. The critical insight was that all distance values must share the same unit (256 per tile), so that the comparisons between `tMaxX` and `tMaxY` remain meaningful regardless of the ray angle. Getting the initial `tMaxX/Y` values (half-step instead of full-step) correct was the most time-consuming debugging task in the renderer.

7.2 INT 1Ch Interrupt Chaining

Replacing the BIOS tick handler rather than chaining to it broke the system clock and prevented the BIOS from calling INT 1Ah's time-of-day service, causing visible drift. The correct pattern — save the original vector before installation and call it via a far jump at the end of every ISR invocation — was non-obvious but essential. Every register,

including **FLAGS**, must be saved and restored because the ISR can fire at any point in the main loop, including in the middle of a multi-instruction sequence.

7.3 Segment Register Management

The double-buffer scheme requires **ES** to point to **CS + 0x1000** during all draw calls and to **0A000h** during the blit, while **DS** must point to the program's data segment at all times in the main loop and to **CS** inside the ISR. Forgetting to restore **DS** after the **rep movsw** blit caused the main loop to silently read wrong data for several frames before a crash. Wrapping the blit in a tight **push DS / pop DS** pair resolved this.

7.4 Enemy AI Edge Cases

The wall follower would occasionally spin in a 2×2 open area (four exits, right turn always available). The braid post-processing pass that removes extra walls and creates loops was added specifically to break these symmetric traps. The smart wanderer oscillated rapidly before the reverse-direction exclusion was added; after excluding the 180° direction from the candidate list the movement became purposeful and visually distinct from the wall follower.

8 Conclusion

8.1 Summary of Achievements

Maze Racer delivers a fully playable first-person 3D maze game in approximately 1,600 lines of x86 assembly. Every major feature maps directly to a COAL curriculum topic: the raycasting engine demonstrates fixed-point arithmetic and memory-mapped I/O; the recursive backtracker demonstrates stack manipulation and conditional branching; the timer ISR demonstrates interrupt handling and register discipline; the PC speaker driver demonstrates hardware port control; and the difficulty system demonstrates bitwise flags and conditional state management. The project is self-contained in a flat DOS binary with no external libraries.

8.2 Limitations

- The double-buffer segment is placed at a fixed offset from **CS** (**CS + 0x1000**), which is fragile if the loader places the program in unusual memory.
- The win screen switches to VGA text mode (**INT 10h / 0Eh**) to print text, disrupting the Mode 13h context; ideally the font system would render the win message within Mode 13h.
- No leaderboard or score persistence across sessions.

8.3 Future Work

- Persistent leaderboard using DOS file I/O (**INT 21h** functions **3Ch/3Fh/40h**).
- Pixel-art animated title screen in Mode 13h before the menu.
- A third enemy type using Manhattan-distance greedy chasing.
- Safe segment allocation via **INT 21h** function **48h** (DOS memory allocation) instead of a fixed offset.

- Sub-tile player position for smooth movement, using a fixed-point position representation.

References

- [1] K. R. Irvine, *Assembly Language for x86 Processors*, 7th ed. Upper Saddle River, NJ, USA: Pearson, 2015.
- [2] R. Hyde, *The Art of Assembly Language*, 2nd ed. San Francisco, CA, USA: No Starch Press, 2010.
- [3] R. Brown and J. Kopplin, “Ralf Brown’s Interrupt List,” Version 61, 2000. [Online]. Available: <http://www.ctyme.com/rbrown.htm>. [Accessed: May 2026].
- [4] OSDev Community, “VGA Hardware,” *OSDev Wiki*, 2024. [Online]. Available: https://wiki.osdev.org/VGA_Hardware. [Accessed: May 2026].
- [5] OSDev Community, “Programmable Interval Timer,” *OSDev Wiki*, 2024. [Online]. Available: https://wiki.osdev.org/Programmable_Interval_Timer. [Accessed: May 2026].
- [6] L. Van den Berg (Lode), “Raycasting,” in *Lode’s Computer Graphics Tutorial*, 2004. [Online]. Available: <https://lodev.org/cgtutor/raycasting.html>. [Accessed: May 2026].
- [7] NASM Development Team, “NASM — The Netwide Assembler,” Version 2.16, 2024. [Online]. Available: <https://www.nasm.us>. [Accessed: May 2026].
- [8] DOSBox Team, “DOSBox,” Version 0.74-3, 2024. [Online]. Available: <https://www.dosbox.com>. [Accessed: May 2026].
- [9] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes in C: The Art of Scientific Computing*, 2nd ed. Cambridge, UK: Cambridge University Press, 1992. (LCG constants referenced in `random_step`.)
- [10] GitHub Repository: <https://github.com/TayyaabZ/Maze-Racer>.